

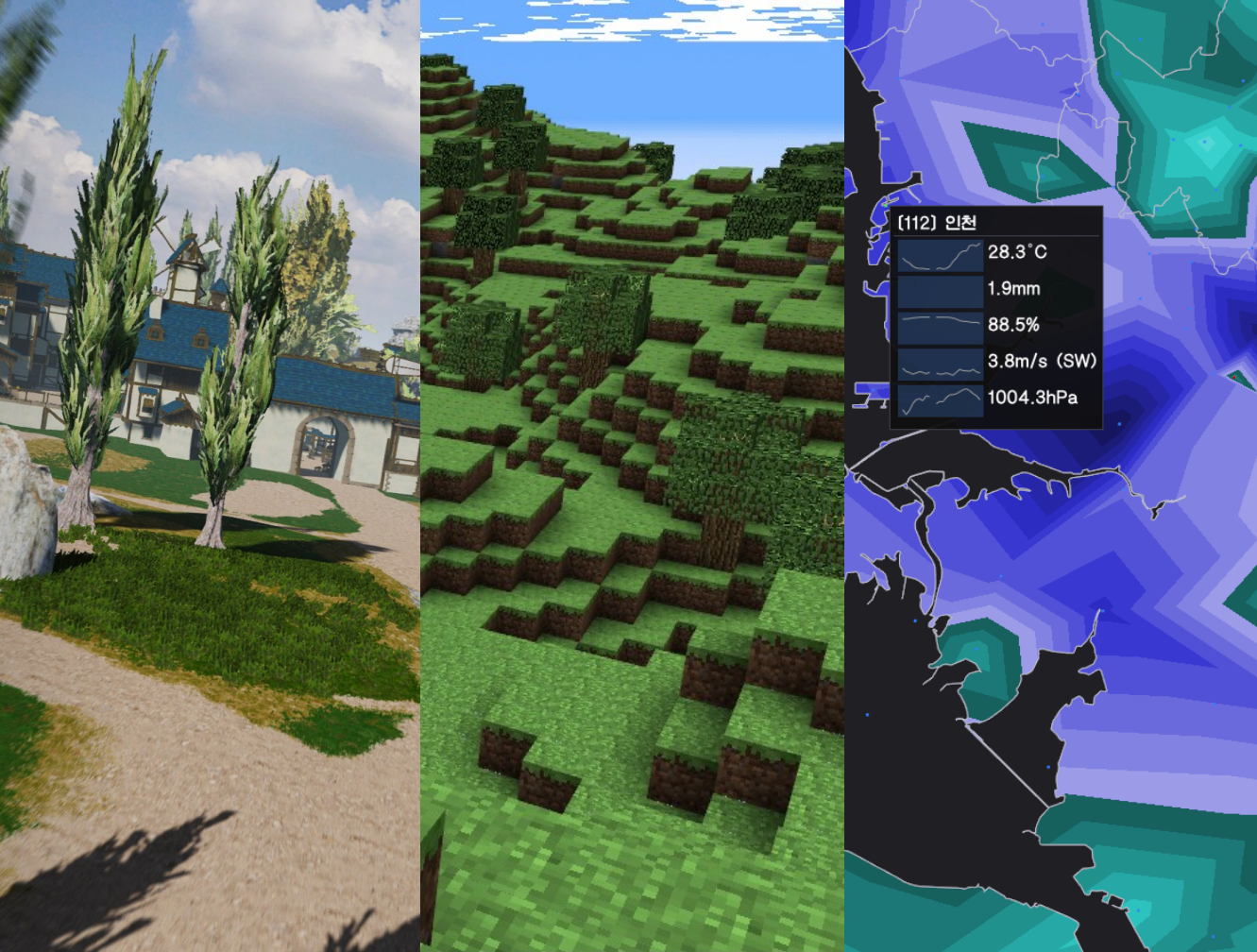
그래픽스 프로그래밍 포트폴리오

김도엽

Tel. : 010 - 2435 - 3404

E-Mail : do.yupdown@gmail.com





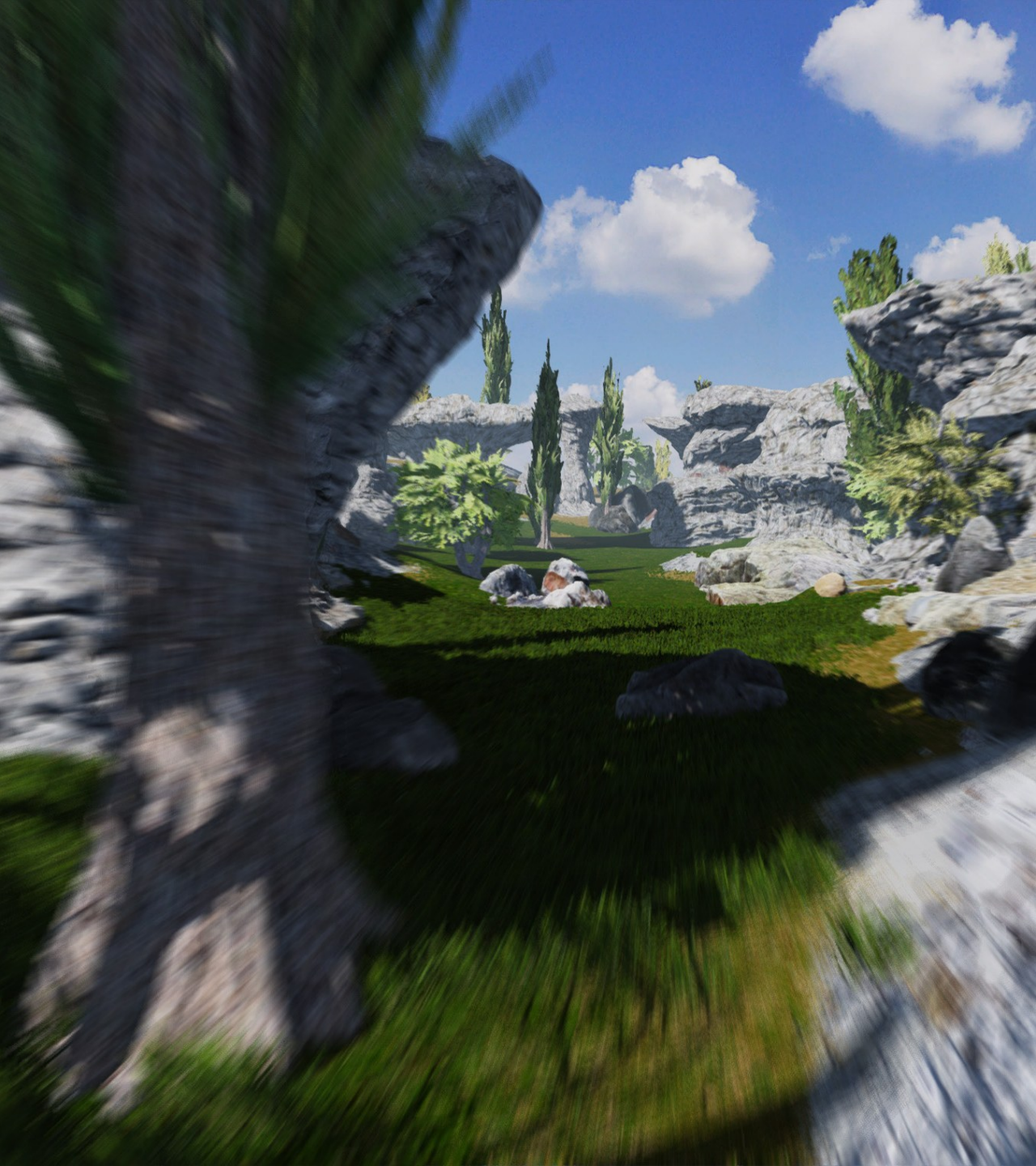
—

PROJECTS

EVERGREEN 게임공학과 졸업작품

CAVE GAME 컴퓨터그래픽스 텀프로젝트

KMETEOROLOGY 스크립트언어 텀프로젝트



EVERGREEN

프로젝트 개요



프로젝트 이름

Evergreen (에버그린)



장르

오픈 월드 판타지 MMORPG

설명

파티퀘스트를 통한 협동 플레이를 중심으로 넓은 세계를 자유롭게 탐험하며 채집 및 사냥하는 게임

개발 언어

C++20, HLSL

사용 툴 & 라이브러리

Visual Studio 2022, DirectX 12, Assimp, IOCP Server, Nsight Graphics

개발 인원

3명 (서버 프로그래밍, 클라이언트 프로그래밍, 캐릭터 모델링)

담당 역할

클라이언트 프로그래밍 - 렌더러 프레임워크 구성, 셰이더 작성, 레벨 파싱, 이펙트, 인터페이스

제작 기간

2024 / 03 ~ 2025 / 09

EVERGREEN

DEVELOPMENT SHOWREEL

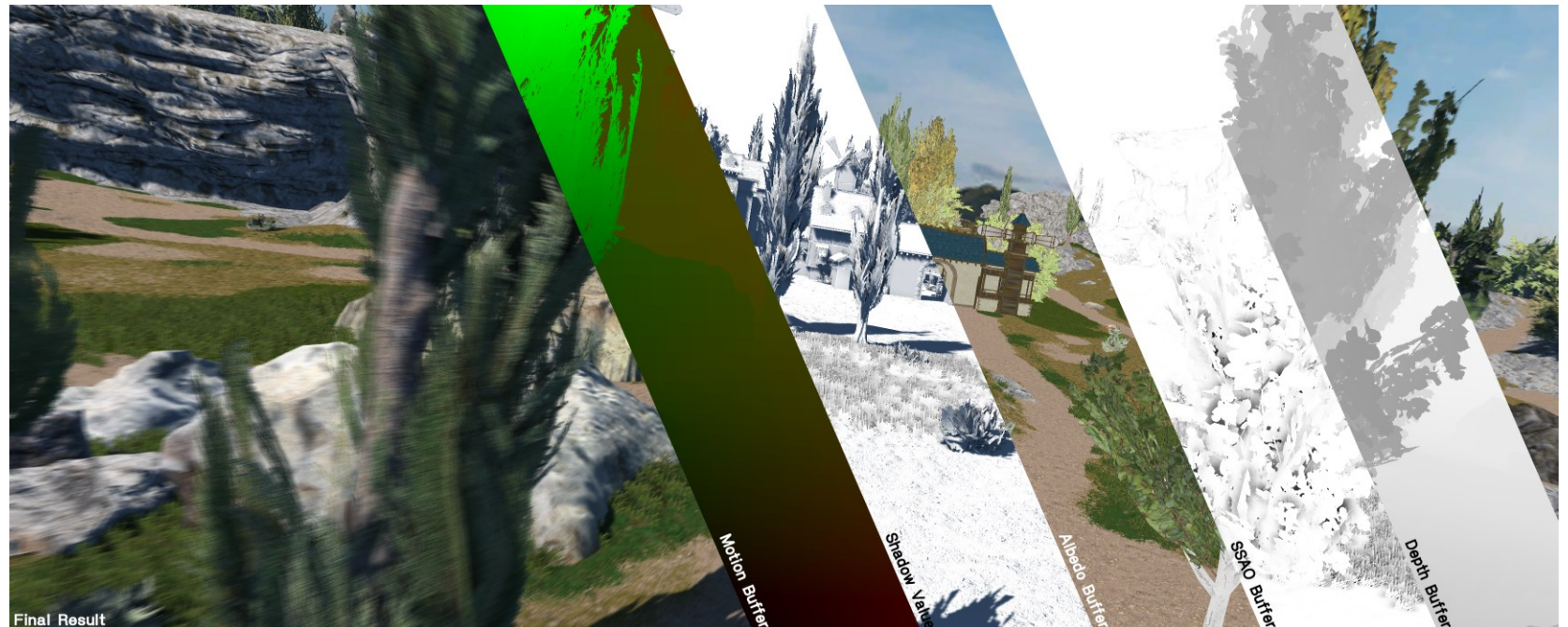


<https://youtu.be/oG4O6Fxbq7A>

DEFERRED HDR RENDER

- ECS (Entity Component System) 기반 DirectX 12 프레임워크
- 3개의 GBuffer를 이용하여 Deferred HDR Render 적용
- Deferred Render 에서 오브젝트마다 서로 다른 매터리얼을 렌더링하기 위해 Stencil Buffer에 셰이더 고유 ID 저장 후 최종 패스에 ID 마다 차례로 렌더
- 캐릭터 렌더링에 NPR 셰이더 적용

```
static constexpr DXGI_FORMAT GBUFFER_FORMATS[NUM_GBUFFERS] = {  
    DXGI_FORMAT_R8G8B8A8_UNORM, // RGB: Albedo color  
    DXGI_FORMAT_R16G16_SNORM,    // RG: Packed View normal vector  
    DXGI_FORMAT_R8G8B8A8_SNORM, // RG: Screenspace motion vector, B: Metallic, A: Roughness  
};
```



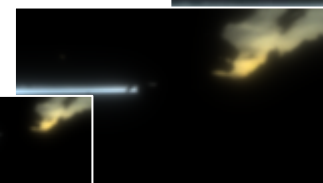
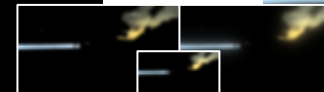
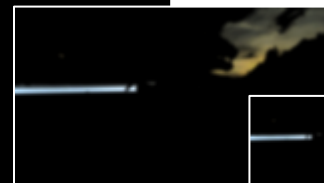
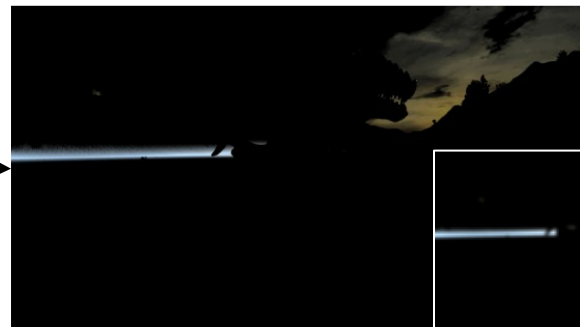
PER-OBJECT MOTION BLUR

- 프레임 간 시간적인 아티팩트를 해결하기 위해 카메라 움직임, 오브젝트 이동 · 회전 · 크기 변화, 캐릭터 애니메이션에 모션 블러 적용
- 이전 프레임의 오브젝트 트랜스폼, 애니메이션 트랜스폼, 카메라 변환 정보를 임시로 저장한 후 화면 공간에서 변위를 구하여 G 버퍼에 저장.
- 후처리 단계에서 변위 정보를 활용하여 가우시안 블러 적용.
- "A reconstruction filter for plausible motion blur" 논문 참고



PHYSICALLY BASED BLOOM

- 밝은 영역에 대한 현실적인 효과를 구성하기 위해 물리 기반 블룸 적용
- R11G11B11_FLOAT 로 저장된 중간 결과 화면의 밝은 영역을 수집하여, 다양한 크기를 가진 여러 개의 버퍼에 걸쳐 밝은 영역을 주변으로 확산
- 확산된 밝은 영역에 대한 버퍼를 결과 화면에 블렌딩 함과 동시에 SEUS Tone Mapping 적용



Convolutions



CSM CASCADED SHADOW MAPS

- 4-levels CSM을 사용하여 가까운 곳은 해상도가 높은 그림자 맵을 사용하면서, 먼 거리까지 그림자를 렌더링
- 거리에 따른 그림자 levels 간 경계를 lerp 함수를 통해 부드럽게 보간
- 부드러운 그림자 경계를 위해 PCF (Percentage Closer Filtering) 적용

```
float ShadowValue(float4 posW, float3 normalW, int level, float bias = 0.0f)
{
    float4 shadowPosH = mul(mul(posW, gLightViewProjClip[level]), gTex);
    // Complete projection by doing division by w.
    shadowPosH.xy /= shadowPosH.w;

    float percentLit = 0.0f;
    if (max(shadowPosH.x, shadowPosH.y) < 1.0f && min(shadowPosH.x, shadowPosH.y) > 0.0f)
    {
        // Depth in HMC space.
        float depth = shadowPosH.z - bias;

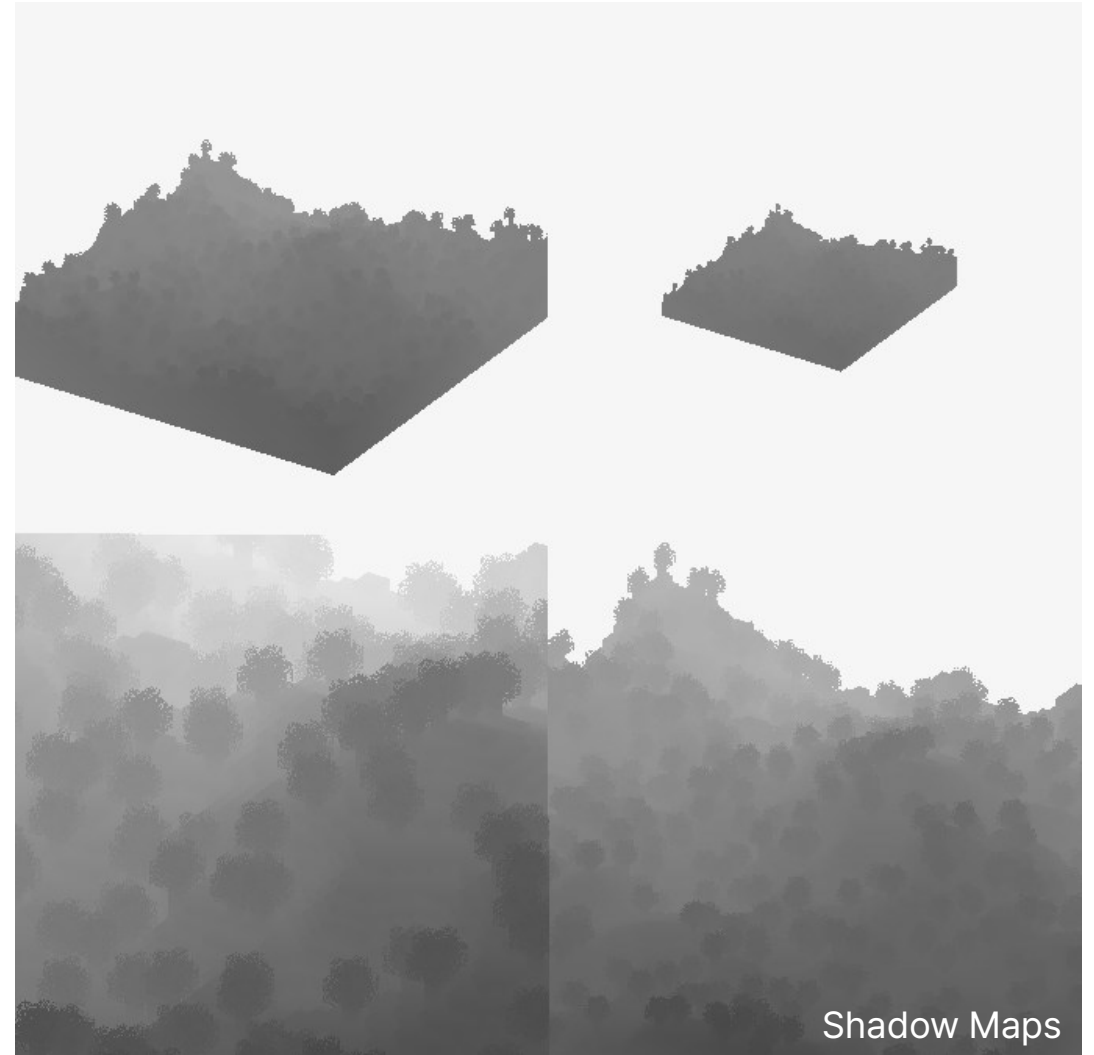
        uint width, height, numTips;
        gShadowMap.GetDimensions(0, width, height, numTips);

        // Texel size.
        float dx = 1.0f / (float)width;
        const float2 offsets[9] = {
            float2(-dx, -dx), float2(0.0f, -dx), float2(dx, -dx),
            float2(-dx, 0.0f), float2(0.0f, 0.0f), float2(dx, 0.0f),
            float2(-dx, +dx), float2(0.0f, +dx), float2(dx, +dx)
        };

        [unroll]
        for (int i = 0; i < 9; ++i)
            percentLit += gShadowMap.SampleCmpLevelZero(gSamplerShadow, shadowPosH.xy + offsets[i], depth).r;
    }
    else
    {
        percentLit = 0.0f;
    }

    return percentLit / 9.0f;
}
```

Shadow 샘플링 셰이더 코드



CAVE GAME



CAVE GAME

프로젝트 개요



프로젝트 이름

Cave Game



장르

오픈 월드 샌드박스 게임

설명

상용 게임 "Minecraft"를 재구현하여 절차적으로 생성된 지형에서 블록을 설치하고 파괴할 수 있는 게임

개발 언어

C++20, GLSL

사용 툴 & 라이브러리

Visual Studio 2022, OpenGL, Assimp

개발 인원

2명 (클라이언트 프로그래밍)

담당 역할

지형 생성, 셰이더 작성, 게임플레이 프로그래밍

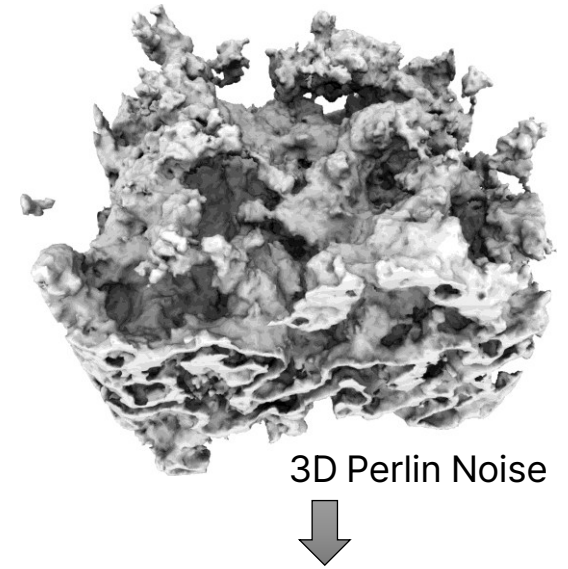
제작 기간

2023 / 09 ~ 2023 / 12

CAVE GAME

절차적 지형 생성

- $1 \times 1 \times 1$ 크기의 정육면체로 이루어진 블록의 집합으로 Voxel 기반 지형 구성
- 여러 단계들로 이루어진 지형 생성 알고리즘을 통해 평지, 언덕, 동굴 등의 다양한 지형을 생성 가능, 절차적 지형 생성에는 3D Perlin Noise 알고리즘 등을 사용
- 각각의 타일은 고유한 텍스처를 가지며, 플레이어가 카메라를 향한 방향의 타일을 Ray Casting 알고리즘을 통해 설치하거나 파괴
- 플레이어를 포함한 모든 오브젝트는 지형과 충돌 시뮬레이션



3D Perlin Noise



```
for (int x = 0; x < MCtilemap::MAP_WIDTH; ++x)
{
    for (int z = 0; z < MCtilemap::MAP_WIDTH; ++z)
    {
        float gLevel = Perlin::Noise((x0 + x) * 0.02f, (z0 + z) * 0.02f) * 16.0f + 12.0f;
        tilemap->SetTile(x, z, 1);
        for (int y = 1; y < MCtilemap::MAP_HEIGHT; ++y)
        {
            float perlinValue = Perlin::Noise((x0 + x) * 0.1f, (y0 + y) * 0.1f, (z0 + z) * 0.1f);
            uint8_t tile = perlinValue * 6.0f + gLevel - y > 0.0f;
            tilemap->SetTile(x, y, z, tile);
        }
    }
}

for (int x = 0; x < MCtilemap::MAP_WIDTH; ++x)
{
    for (int z = 0; z < MCtilemap::MAP_WIDTH; ++z)
    {
        for (int y = 0; y < MCtilemap::MAP_HEIGHT; ++y)
        {
            if (tilemap->GetTile(x, y, z) != 1)
                continue;
            if (y + 1 > MCtilemap::MAP_HEIGHT || tilemap->GetTile(x, y + 1, z) == 0)
                tilemap->SetTile(x, y, z, 0);
        }
    }
}
```

지형 생성 코드



절차적으로 생성된 지형

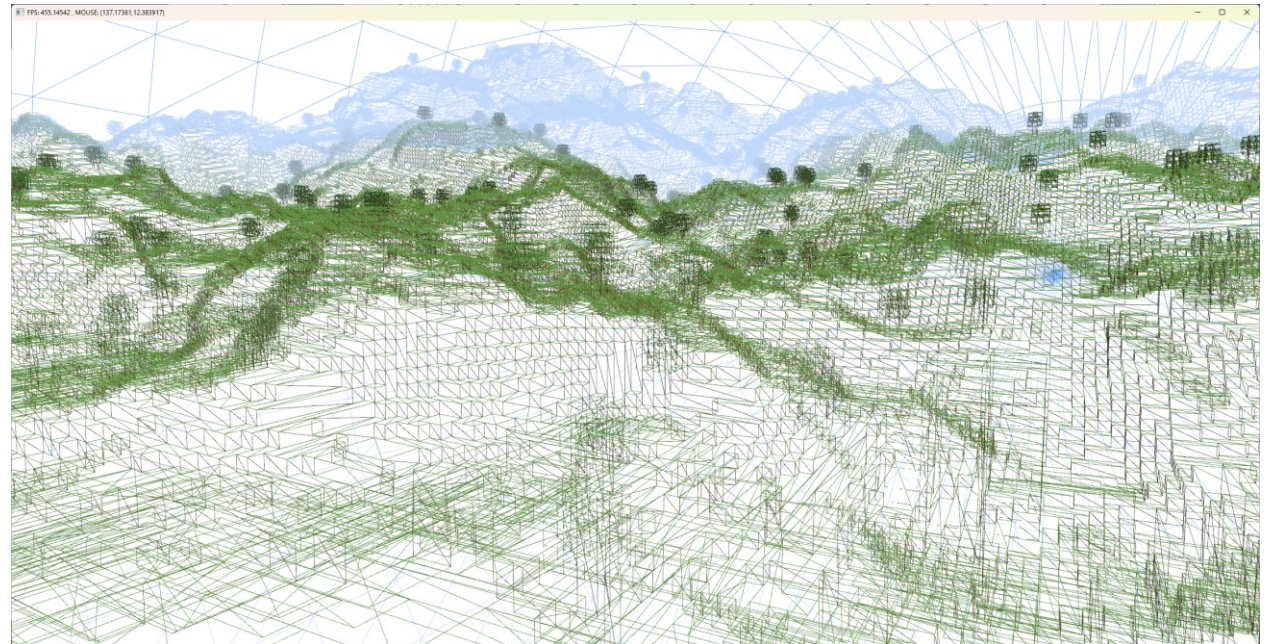
CAVE GAME

렌더링 최적화

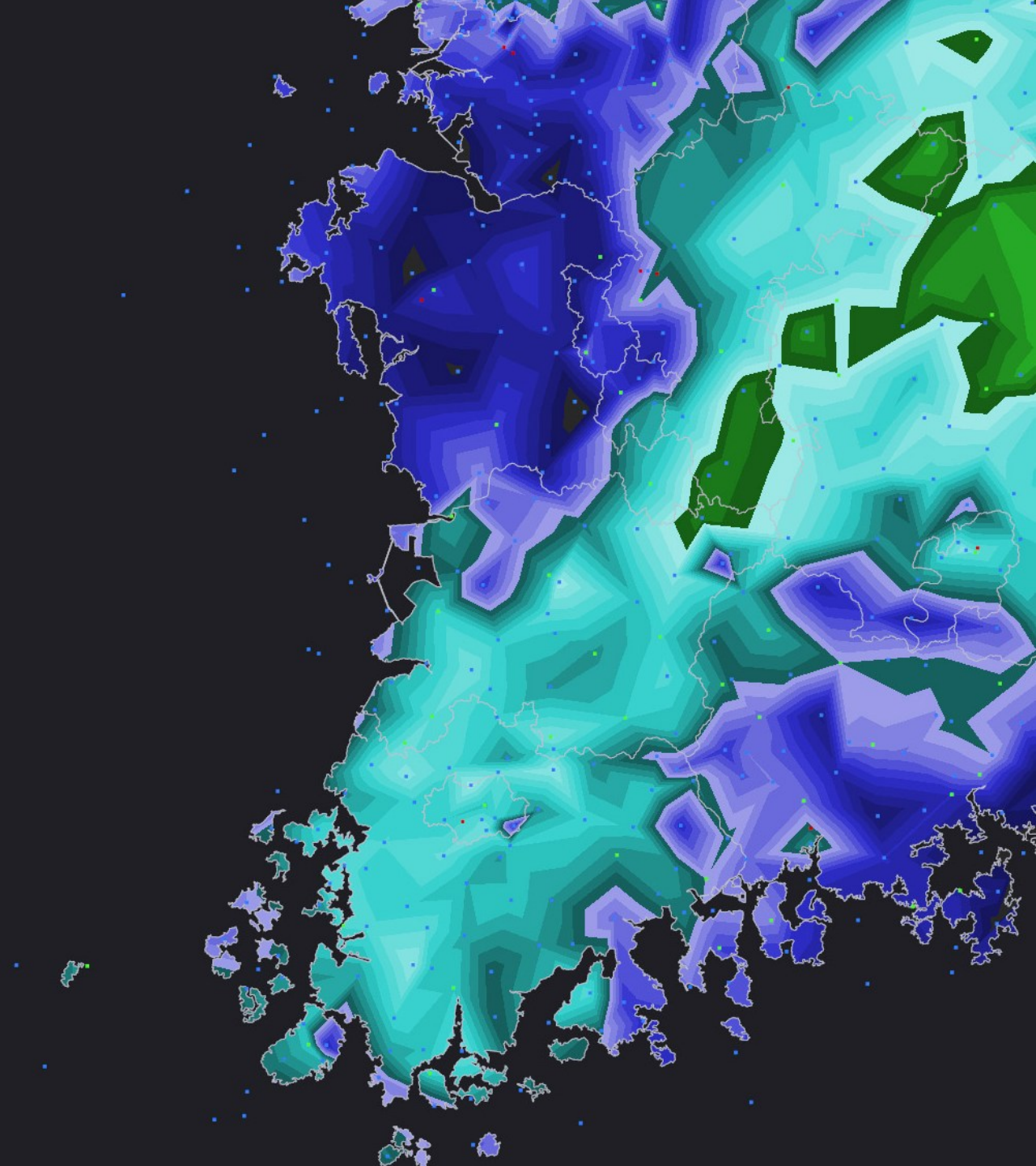
- 지형에서 보이지 않는 면은 버텍스 / 인덱스 버퍼 생성 단계에 폴리곤을 생성하지 않음으로써, 오버드로우 현상을 최소화
- 블록을 설치하거나 파괴할 때, 버텍스 / 인덱스 버퍼를 업데이트하는 오버헤드를 줄이기 위해, 월드를 $32 * 32 * 32$ 블록 크기의 "청크"로 나누어 별도의 버퍼로 관리
- 지형의 드로우 콜을 줄이기 위해 블록들에 대한 다양한 텍스처를 하나로 통합하여, 버텍스의 UV의 값 조작으로 여러 텍스처를 한번의 드로우 콜로 렌더링



통합된 블록 텍스처



지형 와이어프레임



—

KMETEOROLOGY

프로젝트 개요

프로젝트 이름

KMeteorology



설명

기상청 데이터를 이용한 OpenGL 기반 실시간 기상 데이터 시각화

개발 언어

Python 3.12, GLSL

사용 툴 & 라이브러리

PyCharm, OpenGL

개발 인원

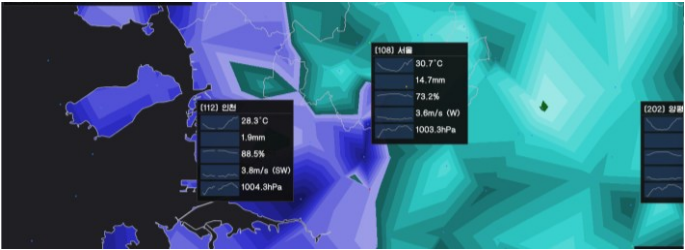
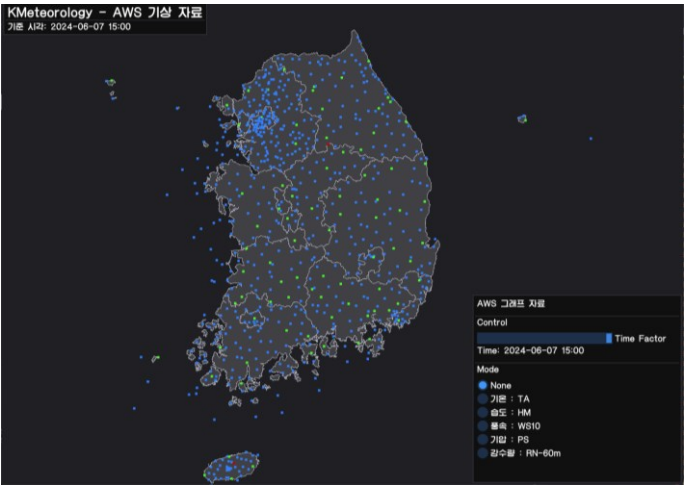
1명

담당 역할

API 를 통한 데이터 수신 및 파싱
벡터 기반 전국 지도 & 히트맵 렌더러 구성

제작 기간

2024 / 03 ~ 2024 / 06



벡터 기반 지도

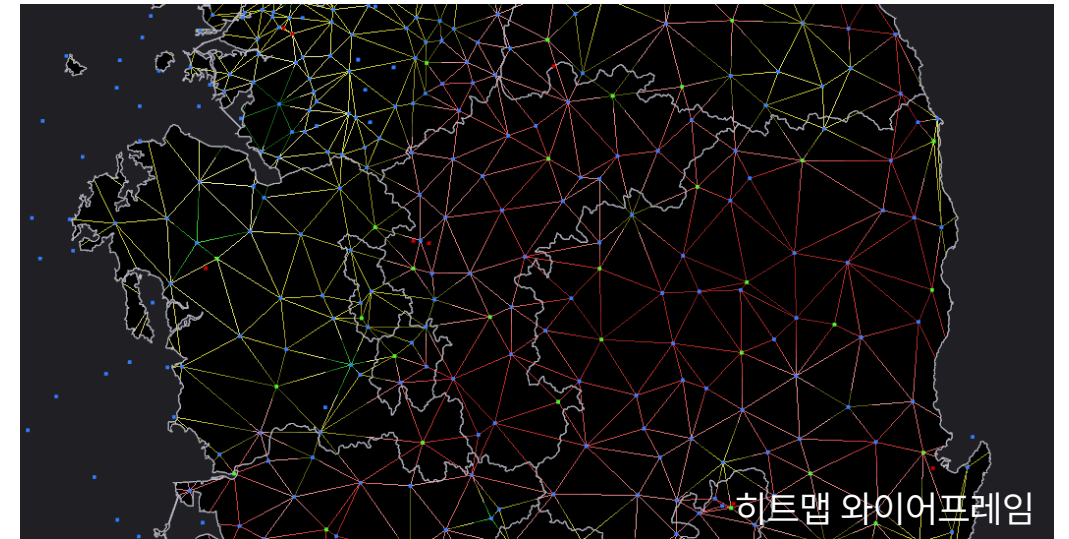
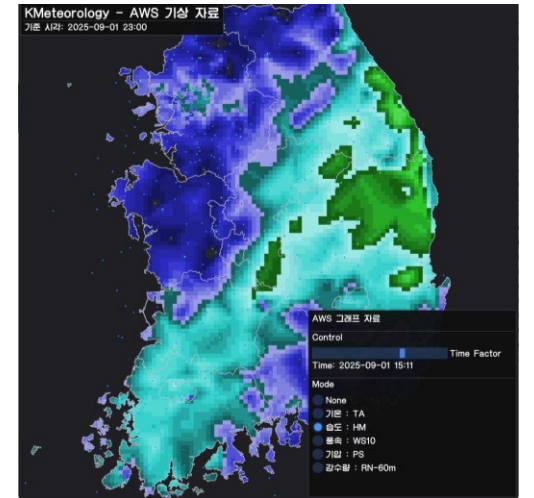
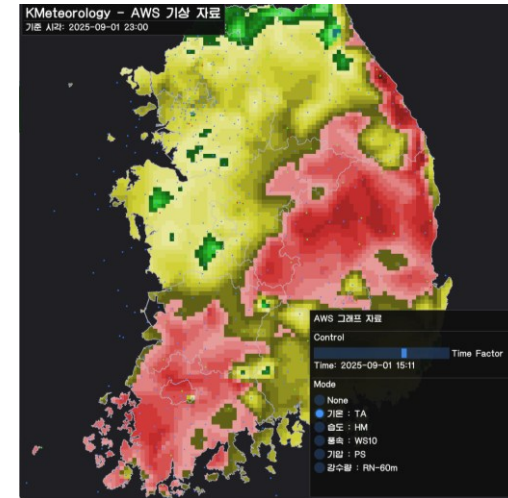
- SVG 형태의 전국 지도를 파싱하여 다각형 정점 리스트를 추출
- 지도의 각 영역은 단순 다각형 (Simple Polygon) 이라는 점을 활용하여 다각형의 한 변과 임의의 점으로 구성된 삼각형을 그렸을 때, 홀수번 덮어 그려진 영역을 최종적으로 렌더
- 스텐실 버퍼의 GL_INVERT OP를 활용하여 그려질 영역에 대한 스텐실 버퍼를 기록하고, 이후 내부 영역을 스텐실 테스트와 함께 렌더
- 영역의 외곽선은 GL_LINE_LOOP로 렌더



히트맵 렌더링

- 각각의 관측소에서 수집된 기상 정보 (기온, 습도, 풍속, 기압, 강수량) 사이의 위치에 대한 정보를 추론 후 시각화하기 위해 주변 3개의 관측소 정보를 선형 보간
- 각 관측소의 위경도를 지도 공간에 매핑한 정점 집합에 대해서 들로네 삼각분할 알고리즘으로 생성된 삼각형에 대한 인덱스 버퍼 생성
- 각 관측소에 대응되는 정점에 기상 정보값을 VBO를 통해 GPU에 전송
- 래스터라이저에 보간된 픽셀 위치의 기상 정보값을 1D 컬러 팔레트 텍스처에 매핑하여 히트맵 렌더

1D 컬러 팔레트 텍스처





THANK YOU

